Genome Assemblies

How do we assemble genomes?

- Determining the order of nucleotides in a genome, or **genome sequencing**, presents a fundamental task in bioinformatics
- Genomes vary in length; genome is roughly 3 billion nucleotides long, whereas the genome of *Amoeba dubia*, an amorphous unicellular organism, is approximately 200 times longer!
- This unicellular organism competes with the rare Japanese flower *Paris japonica* for the title of species with the longest genome
- The first sequenced genome, belonging to a ϕX174 bacterial phage (i.e., a virus that preys on bacteria), had only 5,386 nucleotides and was completed in 1977 by Frederick Sanger.

Genome race…

- Four decades after this Nobel Prize-winning discovery, genome sequencing has raced to the forefront of bioinformatics research, as the cost of sequencing plummeted
- Because of the decreasing cost of sequencing, we now have thousands of sequenced genomes, including those of many mammals

1. The largest obstacle is the fact that biologists still lack the technology to read the nucleotides of a genome from beginning to end in the same way that you would read a book
2. The best they can do is sequence much shorter DNA fragments called **reads**

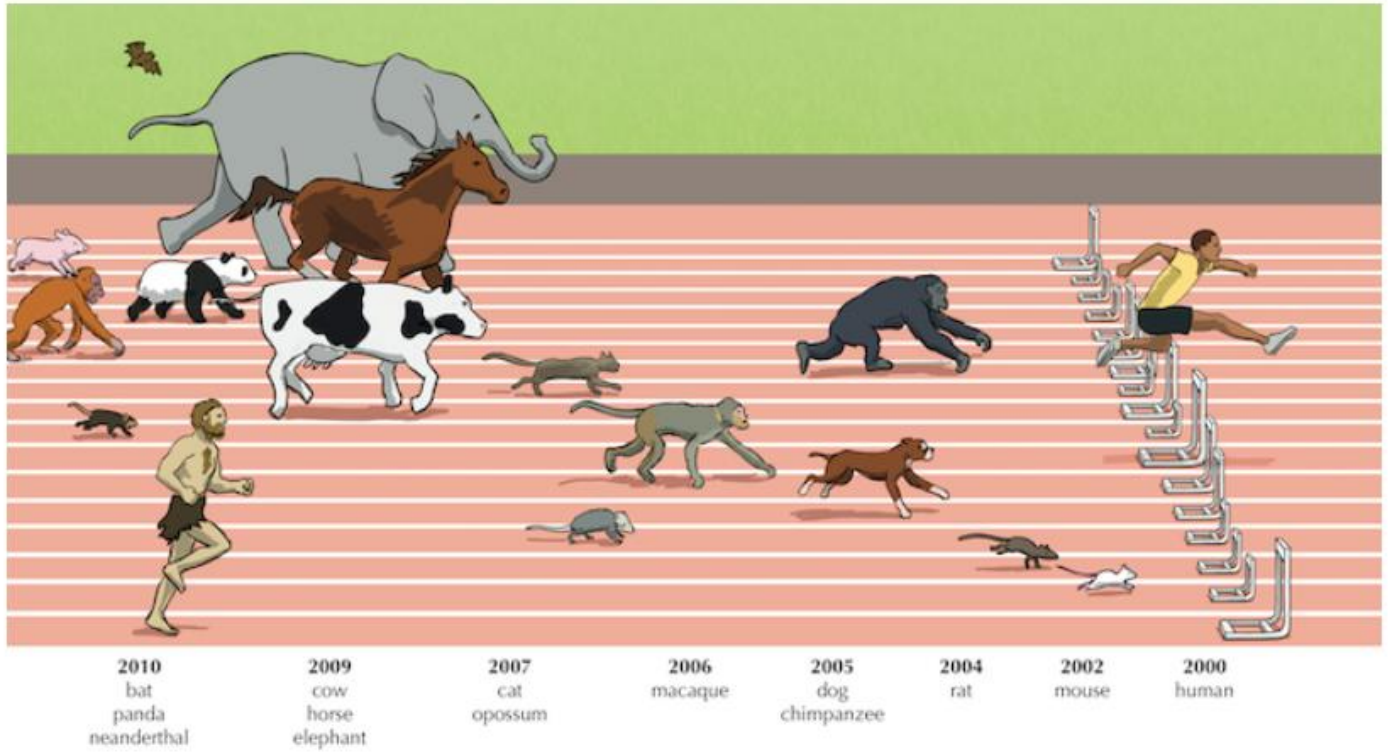


Figure: The first mammals with sequenced genomes.

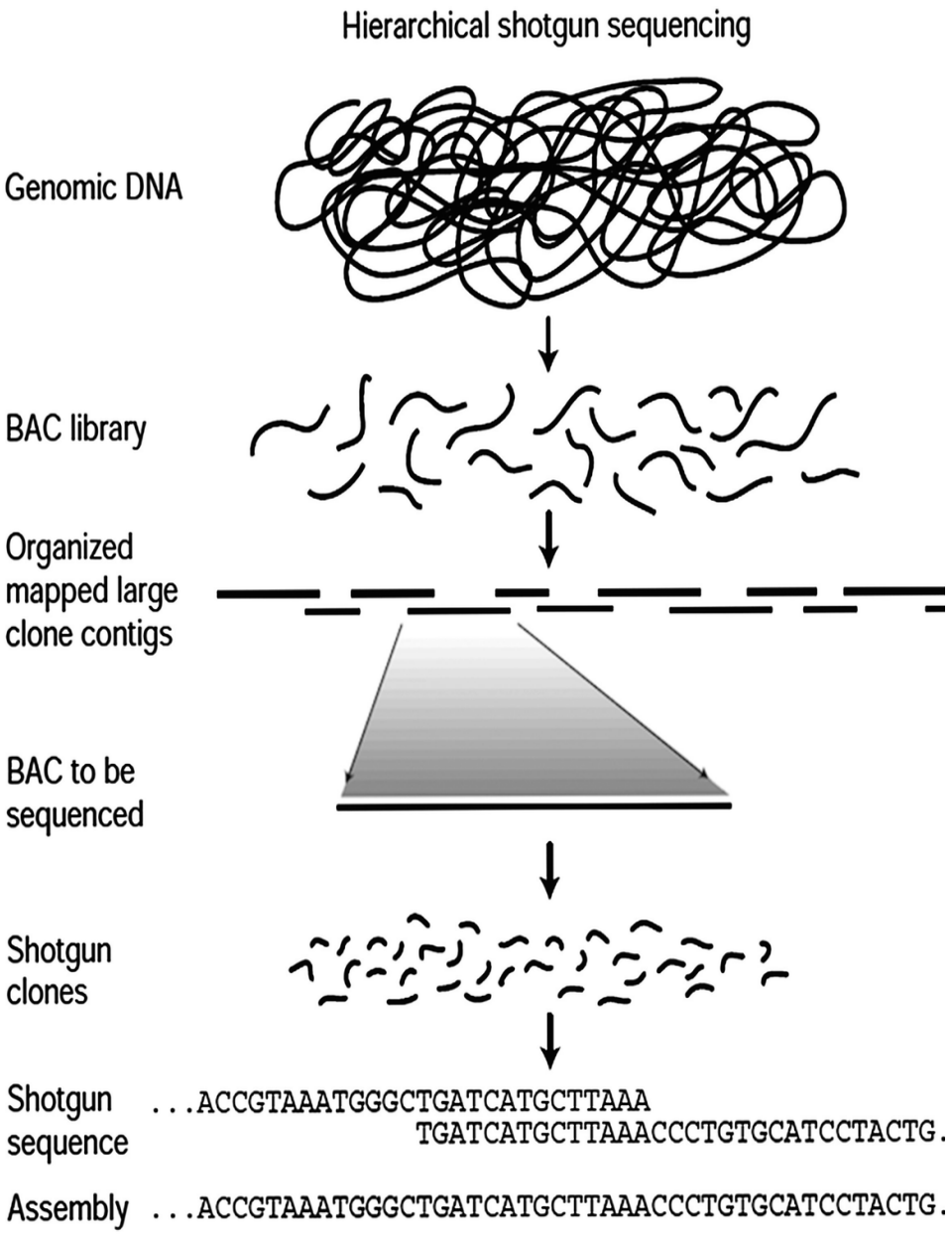
How does the human genome stack up?

Organism Genome	Size (Bases)	Estimated Genes
Human (Homo sapiens)	3 billion	30,000
Laboratory mouse (M. musculus)	2.6 billion	30,000
Mustard weed (A. thaliana)	100 million	25,000
Roundworm (C. elegans)	97 million	19,000
Fruit fly (D. melanogaster)	137 million	13,000
Yeast (S. cerevisiae)	12.1 million	6,000
Bacterium (E. coli)	4.6 million	3,200
Human immunodeficiency virus (HIV)	9700	9

The Human Genome Project (HGP)

- The $3 BILLION publicly funded HGP lasted 13 years.
- Initiated in 1990
- Declared complete on April 14, 2003 [final sequence mapping]
- Key principles
  - Building genetic and physical maps of the human and mouse genomes
  - Sequencing the smaller yeast and worm genomes as a test run for sequencing the larger, more complex human genome (IHGSC, 2001).
- Two-Phase approach to tackle the Human Genome Sequence
  - Shotgun Phase
  - Finishing Phase

Chial, Heidi. "DNA sequencing technologies key to the Human Genome Project." *Nature Education* 1.1 (2008).



- The finished sequence covered 99% of the human gene containing regions with 2.85 billion nucleotides of the genome.
- A predicted error rate of 1 event per 100,000 bases sequenced.
- The number of gaps was reduced by 400-fold; only 349 gaps out of 147,821 gaps remained.

Chial, Heidi. "DNA sequencing technologies key to the Human Genome Project." *Nature Education* 1.1 (2008).

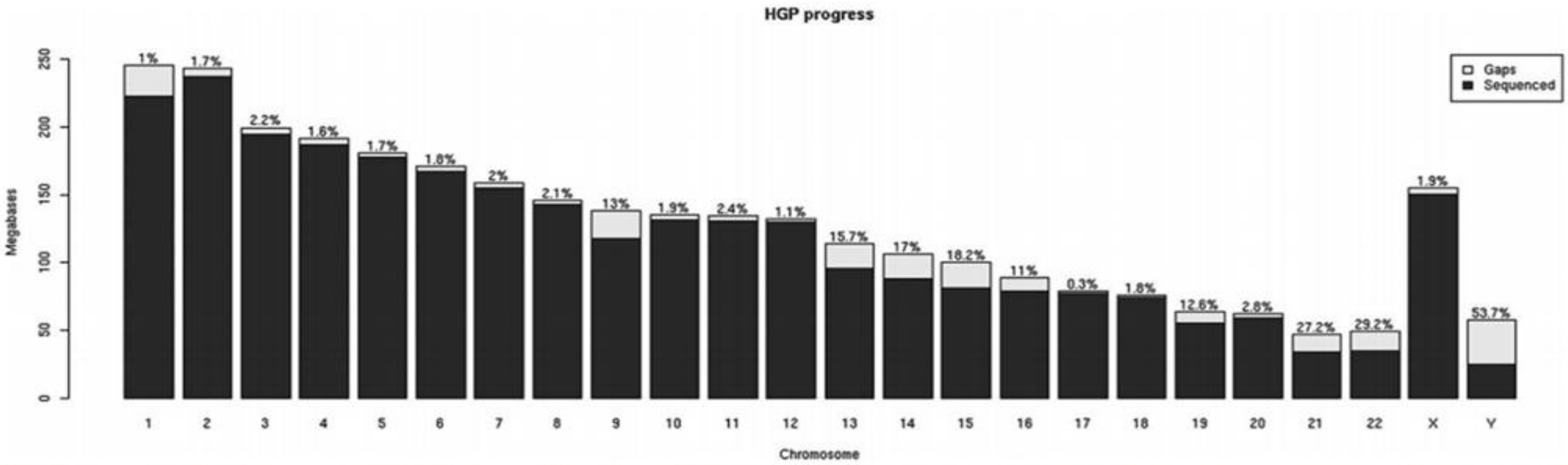


Figure The sequenced (gray filled) and unsequenced (white) portions of the human genome, listed by chromosome; numbers in % are the proportion of chromosomes that are heterochromatic and unsequenced for this reason. Statistics are from the NCBI Build 35, UCSC assembly of May 2004, Assembly 17, data from http://www.genome.ucsc.edu/goldenPath/stats.html#hg17.

Homo sapiens GRCh38.p13	
Number of regions with alternate loci or patches	358
Total sequence length	3,099,706,404
Total ungapped length	2,948,583,725

median total length (Mb): 2893.5
median protein count: 121425
median GC%: 40.9

Genome Reference Consortium Human Build 38 patch release 13 (GRCh38.p13)
https://www.ncbi.nlm.nih.gov/assembly/GCF_000001405.39

Large-scale structures and Genetic variation in the genome

- Human DNA is comprised of abundant interspersed repeats- 45% of our DNA consists of repetitive elements interspersed within nonrepetitive sequences.
- Interestingly, the extent and diversity of gene repetitions contained in low copy number repeats were greater than expected
- Extending the initial analysis, full duplication landscape of the genome, shows that there are significant copy number polymorphisms between individuals, the phenotypic consequences of which in many cases are unknown. (CNV, STR, VNTR)
- An insertion/deletion polymorphism, commonly abbreviated “indel,” is a type of genetic variation in which a specific nucleotide sequence is present (insertion) or absent (deletion).
- More than 3 million human genetic variations, called single nucleotide polymorphisms (SNPs) were found.

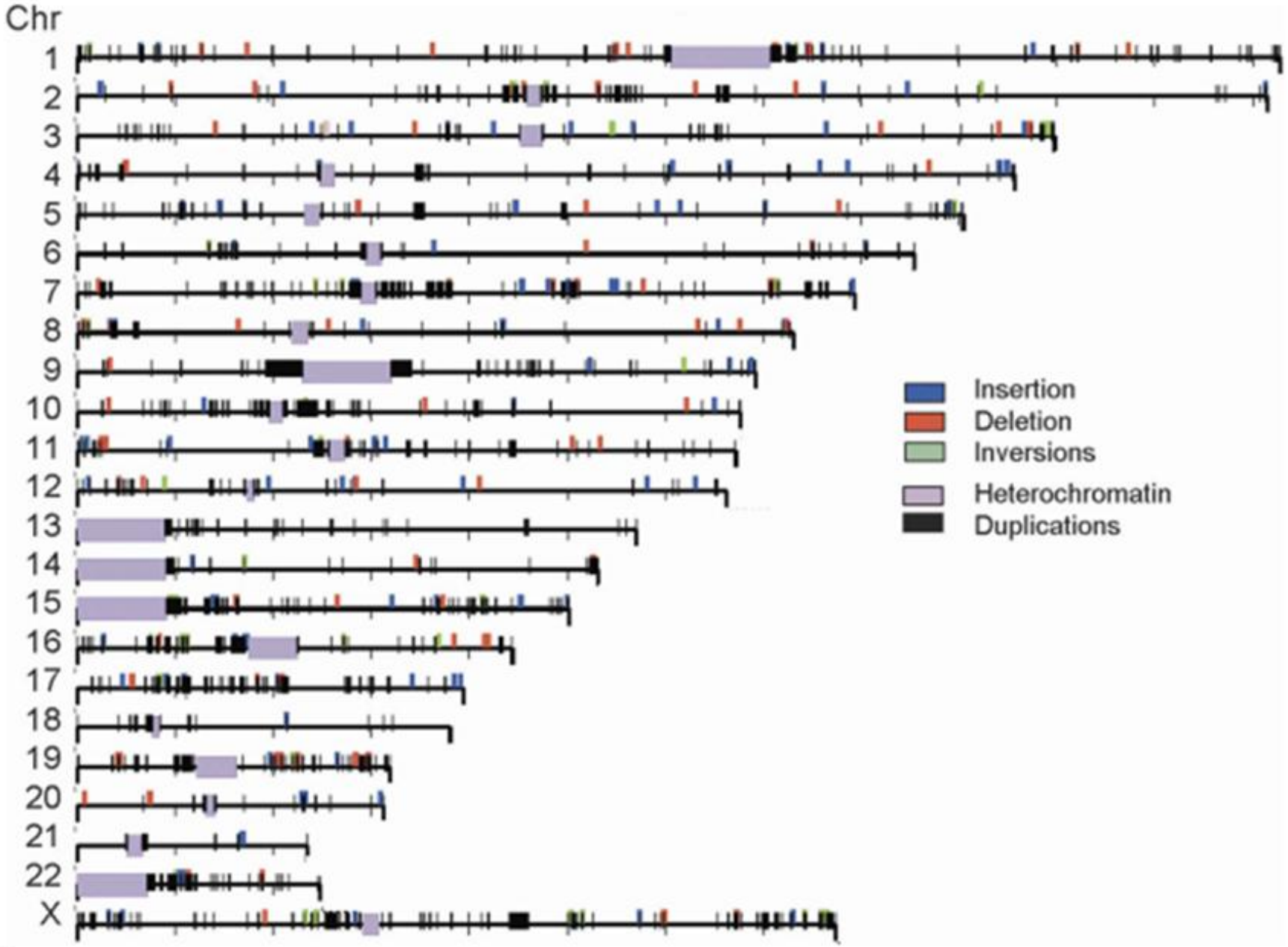
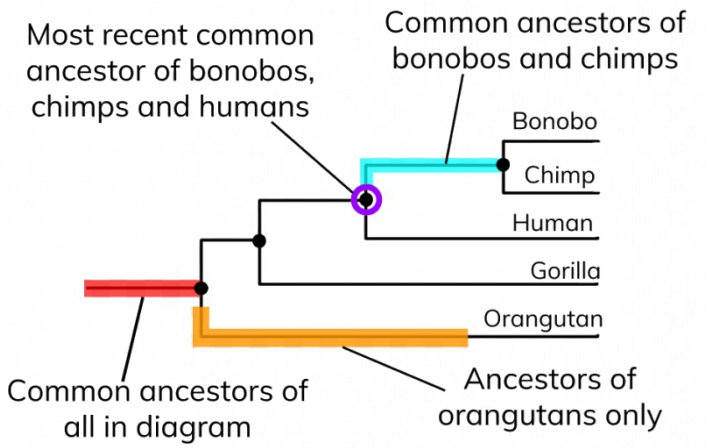


Figure Location of 139 insertions, 102 deletions, and 56 inversions on each human chromosome, showing the location positioned against the DNA sequence. Reproduced and modified with permission from *Nature Genetics* © 2005, Tuzun et al. (2005).

Recreating Human Ancestry



Human Evolution Phylogenetic Tree Diagram
Organism (Joel_Bentzinger)

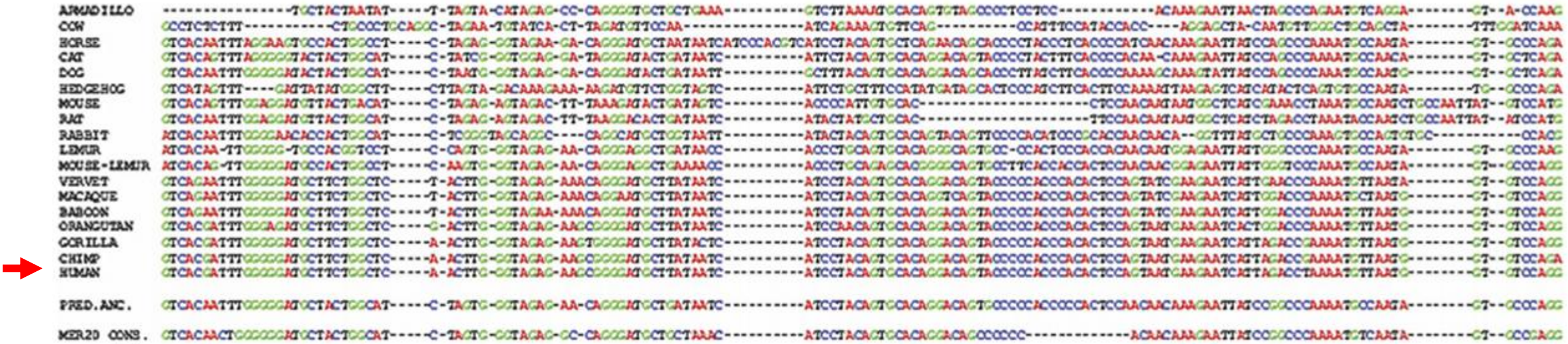
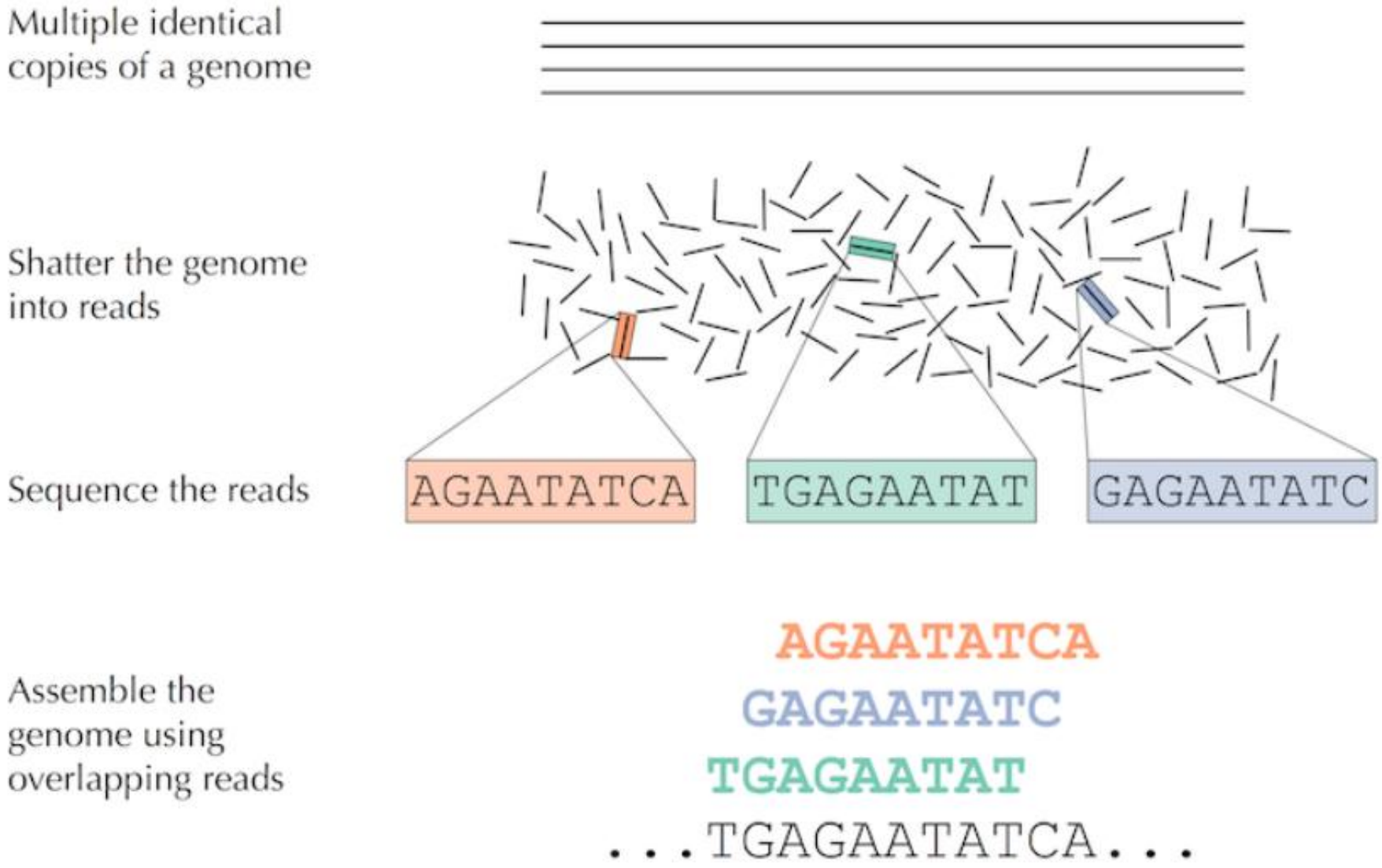


Figure The recreation of an ancestral DNA sequence. This is an example based upon a MER20 retrotransposon. The sequence labeled "PRED.ANC." at the *bottom* is the prediction of the ancestral MER20 sequence. Reproduced and modified with permission from Cold Spring Harbor Laboratory Press © 2004, Blanchette et al. (2004).

Genome Assembly

- The traditional method for sequencing genomes- Researchers take a small tissue or blood sample containing millions of cells with identical DNA, use biochemical methods to break the DNA into fragments, and then sequence these fragments to produce reads
- The difficulty is that researchers do not know where in the genome these reads came from, and so they must use overlapping reads to reconstruct the genome. Thus, putting a genome back together from its reads, **or genome assembly**



In DNA sequencing, many identical copies of a genome are broken in random locations to generate short reads, which are then sequenced and assembled into the nucleotide sequence of the genome

- Even though researchers have sequenced many genomes, a giant genome like that of *Amoeba dubia* still remains beyond the reach of modern sequencing technologies
- the barrier to sequencing such a genome would be experimental, but that is not true; biologists can easily generate enough reads to analyze a large genome, but assembling these reads still presents a major computational challenge

The String Reconstruction Problem

- Genome assembly is more difficult than you think
- DNA is double-stranded, and we cannot determine in advance which strand a specific read originates from, leaving us uncertain whether to use the read or its reverse complement when assembling a particular strand of the genome
- Additionally, modern sequencing machines are prone to errors, and the reads they produce often contain inaccuracies
- These sequencing errors pose challenges for genome assembly by hindering the identification of all overlapping reads
- Furthermore, certain regions of the genome may lack coverage entirely, making it impossible to reconstruct the complete genome

The String Reconstruction Problem

- Modern sequencers typically produce reads of uniform length, allowing us to assume that all reads are k-mers for a specific value of k
- In the case, we will consider an ideal—and unrealistic—scenario where all reads originate from the same strand, are error-free, and provide perfect coverage, ensuring that every k-mer substring of the genome is represented as a read
- Later, we will explore how to adjust these assumptions to accommodate more realistic datasets

Reconstructing strings from *k*-mers

- We are ready to define a computational problem modeling genome assembly
- Given a string Text, its k-mer composition $\text{Composition}_{\mathbf{k}}$(Text) is the collection of all k-mer substrings of Text (including repeated k-mers).
- For example,
  - $\text{Composition}_{\mathbf{3}}$(TATGGGGTGC) = {ATG, GGG, GGG, GGT, GTG, TAT, TGC, TGG}
- We have listed k-mers in lexicographic order (i.e., how they would appear in a dictionary) rather than in the order of their appearance in TATGGGGTGC, because the correct ordering of the reads is unknown when they are generated
- **String Composition Problem:** *Generate the k-mer composition of a string.*
  - **Input:** An integer *k* and a string *Text*.
  - **Output:** $Composition_k(Text)$, where the *k*-mers are arranged in lexicographic order.

Problem

- Code Challenge: Solve the String Composition Problem.
  - Input: An integer k and a string Text.
  - Output: $\text{Composition}_{\mathbf{k}}(\text{Text})$.

- String Reconstruction Problem: Reconstruct a string from its k-mer composition.
  - Input: An integer k and a collection Patterns of k-mers.
  - Output: A string Text with k-mer composition equal to Patterns (if such a string exists).

The String Reconstruction Problem- example

- Let's consider the following example of a 3-mer composition:

  AAT ATG GTT TAA TGT

- The most natural way to solve the String Reconstruction Problem is to mimic the solution and "connect" a pair of k-mers if they overlap in k-1 symbols

- For the above example, it is easy to see that the string should start with **TAA** because there is no 3-mer ending in **TA**

- This implies that the next 3-mer in the string should start with **AA**. There is only one 3-mer satisfying this condition, **AAT**:

  TAA AAT

- In turn, AAT can only be extended by ATG, which can only be extended by TGT, and so on, leading us to reconstruct TAATGTT:

Additional Challenge

- Let's consider an additional 3-mer composition:

AAT ATG ATG ATG CAT CCA GAT GCC GGA GGG GTT TAA TGC TGG TGT

- If we start again with TAA, then the next 3-mer in the string should start with AA, and there is only one such 3-mer, AAT. In turn, AAT can only be extended by ATG

```
TAA
 AAT
  ATG
TAATG
```

- ATG can be extended either by TGC, or TGG, or TGT. Now we must decide which of these 3-mers to choose. Let's select TGT

```
TAA
 AAT
  ATG
   TGT
TAATGT
```

After TGT, our only choice is GTT

```
TAA
 AAT
  ATG
   TGT
    GTT
TAATGTT
```

- Unfortunately, now we are stuck at GTT because no 3-mers in the composition start with TT! We could try to extend TAA to the left, but no 3-mers in the composition end with TA
- You may have found this trap on your own and already discovered how to escape it. If you think a few steps ahead, then you would never extend ATG by TGT until reaching the end of the genome
- With this thought in mind, let's take a step back, extending ATG by TGC instead

```
TAA
 AAT
  ATG
   TGC
TAATGC
```

- Continuing this process…..

```
TAA
 AAT
  ATG
   TGC
    GCC
     CCA
      CAT
       ATG
        TGG
         GGA
          GAT
           ATG
            TGT
             GTT
TAATGCCATGGATGTT
```

Yet this assembly is incorrect because we have only used fourteen of the fifteen 3-mers in the composition (we omitted GGG), making our reconstructed genome one nucleotide too short

Repeats complicate genome assembly

- The difficulty in assembling this simulated genome arises because **ATG** is repeated three times in the 3-mer composition, which causes us to have the three choices **TGG**, **TGC**, and **TGT** by which to extend **ATG**
- Repeated substrings in the genome are not a serious problem when we have just 15 reads, but with millions of reads, repeats make it much more difficult to "look ahead" and construct the correct assembly
- If you recall our discussion on identifying the origin of replication in bacterial genomes, you'll understand how rare it is to observe a long repeat in a randomly generated nucleotide sequence. However, real genomes are far from random
- In fact, around 50% of the human genome consists of repeats
- For example, the Alu sequence, which is approximately 300 nucleotides long, is repeated over a million times, with only minor variations such as nucleotide insertions, deletions, or substitutions

String Reconstruction as a Walk in the Overlap Graph

- From a string to a graph
- Repeats in a genome necessitate some way of looking ahead to see the correct assembly in advance
  - **TA**ATGCCATGGGATGTT is a solution to the String Reconstruction Problem for the collection of fifteen 3-mers
  - We use a different color for each interval of the string between occurrences of ATG
  - Is this the only solution to the String Reconstruction Problem for this collection of 3-mers?

```
TAA
 AAT
  ATG
   TGC
    GCC
     CCA
      CAT
       ATG
        TGG
         GGG
          GGA
           GAT
            ATG
             TGT
              GTT
TAATGCCATGGGATGTT
```

Continued..

In the figure below, consecutive 3-mers in **TAATGCCATGGGATGTT** are linked together to form this string's **genome path**.

Figure: The fifteen color-coded 3-mers making up **TAATGCCATGGGATGTT** are joined into the genome path according to their order in the genome

String Spelled by a Genome Path Problem. *Reconstruct a string from its genome path.*

- **Input:** A sequence *path* of *k*-mers $Pattern_1, \ldots, Pattern_n$ such that the last k - 1 symbols of $Pattern_i$ are equal to the first k-1 symbols of $Pattern_{i+1}$ for $1 \le i \le n$-1.
- **Output:** A string *Text* of length $k+n-1$ such that the i-th k-mer in *Text* is equal to $Pattern_i$ (for $1 \le i \le n$).

Could you construct the genome path if you knew only the genome's 3-mer composition?

Continued…

- We will use the terms **prefix** and **suffix** to refer to the first $k-1$ nucleotides and last $k-1$ nucleotides of a k-mer, respectively

For example, *Prefix*(**TAA**) = **TA** and *Suffix*(**TAA**) = **AA**

Note that the suffix of a 3-mer in the genome path is equal to the prefix of the following 3-mer in the path. For example, *Suffix*(**TAA**) = *Prefix*(**AAT**) = **AA** in the genome path for **TAATGCCATGGGATGTT**

This suggests a method of constructing a string's genome path from its k-mer composition: we will use an arrow to connect any k-mer *Pattern* to a k-mer *Pattern'* if the suffix of *Pattern* is equal to the prefix of *Pattern'*

Problem statement

- Apply the rule to the 3-mer composition of **TA**ATGCCATGGGATGTT. Are you able to reconstruct the genome path for **TA**ATGCCATGGGATGTT?
- If we strictly follow the rule of connecting two 3-mers with an arrow every time the suffix of one is equal to the prefix of the other, then we will connect all consecutive 3-mers in **TA**ATGCCATGGGATGTT
- However, because we don't know this genome in advance, we wind up having to connect many other pairs of 3-mers as well
- For example, each of the three occurrences of ATG should be connected to TGC, TGG, and TGT, as shown

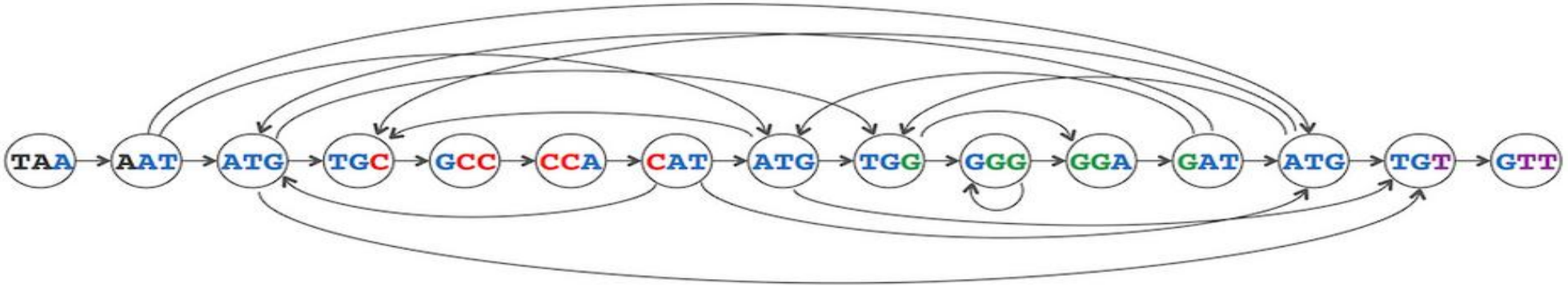


Figure: The graph showing all connections between nodes representing the 3-mer composition of **TA**ATGCCATGGGATGTT.

This graph has fifteen nodes and 28 edges. Note that the genome can still be spelled out by walking along the horizontal edges from **TA**A to GTT

Continued…

- Thus, the figure presents an example of a **graph**, or a network of **nodes** connected by **edges**
- This particular graph is an example of a **directed graph**, whose edges have a direction and are represented by arrows (as opposed to **undirected graphs** whose edges do not have directions)
- But in genome sequencing, we do not know in advance how to correctly order reads. Therefore, we will arrange the 3-mers lexicographically, which produces the overlap graph shown in the figure below

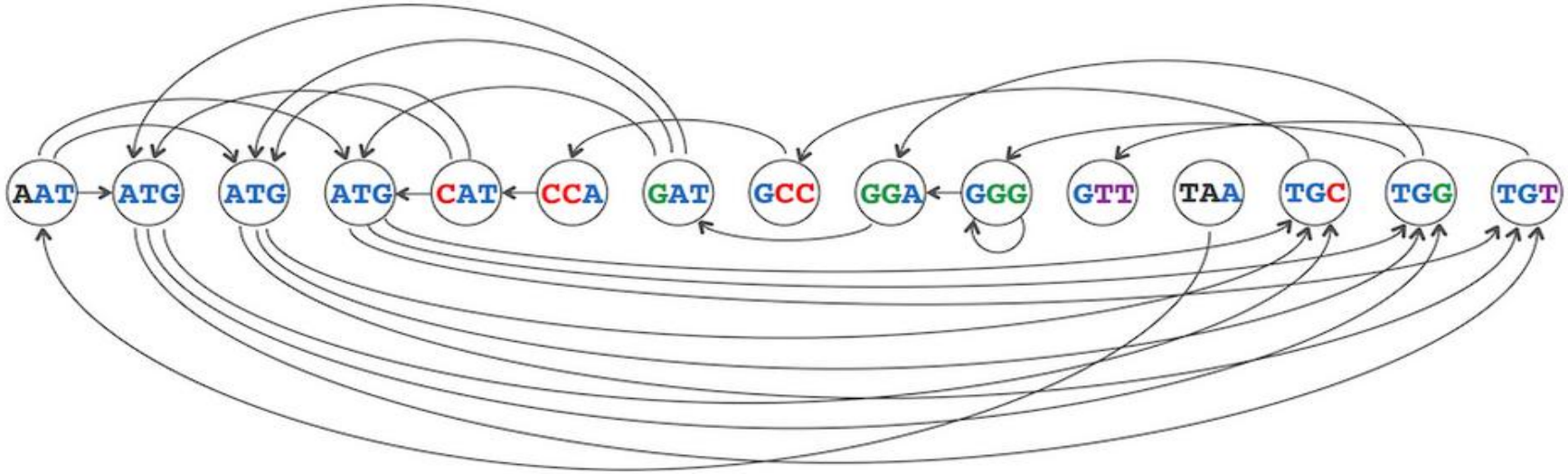


Figure: The same graph as the one in the figure at the top of the page, with 3-mers ordered lexicographically. The path through the graph representing the correct assembly is now harder to see.

The genome vanishes

- We are referring to a genome assembly graph or a sequence graph where paths represent possible genome reconstructions. If a particular genome path appears to have "disappeared," but the nodes are simply rearranged, the path likely still exists but in a different form

- This could happen in several ways:

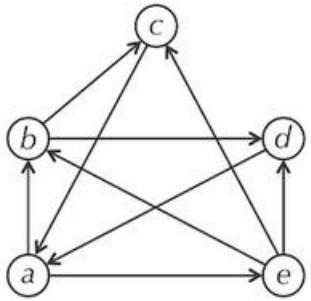
  1. **Graph Representation Change** – If the layout of the graph has changed (e.g., a different traversal algorithm or visualization method), the path may no longer be obvious, but it still exists in the data structure
  2. **Compression or Simplification** – Some genome assembly tools simplify graphs by merging identical sequences or resolving bubbles, which might make a previously obvious path less apparent
  3. **Alternative Paths** – If the graph structure was modified (e.g., different edge weights or a different traversal method), the preferred path might have changed, but the original one is still present in the topology

Overlap Graph Problem

Construct the overlap graph of a collection of k-mers.

- **Input:** A collection Patterns of *k*-mers.
- **Output:** The overlap graph *Overlap*(*Patterns*)
- There are two standard ways of representing a graph
- For a directed graph with *n* nodes, the $n \times n$ **adjacency matrix** ($A_{i,j}$) is defined by the following rule: $A_{i,j} = 1$ if a directed edge connects node *i* to node *j*, and $A_{i,j} = 0$ otherwise
- Another way of representing a graph is to use an **adjacency list**, for which we simply list all nodes connected to each node

Graph



Adjacency Matrix

	a	b	c	d	e
a	0	1	0	0	1
b	0	0	1	1	0
c	1	0	0	0	0
d	1	0	0	0	0
e	0	1	1	1	0

Adjacency List

a is adjacent to *b* and *e*
b is adjacent to *c* and *d*
c is adjacent to *a*
d is adjacent to *a*
e is adjacent to *b*, *c*, and *d*

Figure: A graph with five nodes and nine edges, followed by its adjacency matrix and adjacency list

Solve the Overlap Graph Problem

- **Input:** A collection *Patterns* of *k*-mers.
- **Output:** The overlap graph *Overlap*(*Patterns*), in the form of an adjacency list. (You may return the nodes and their edges in any order.)
- We are looking for a path in the overlap graph that visits every node exactly once
- A path in a graph visiting every node once is called a **Hamiltonian path**, in honor of the Irish mathematician William Hamilton

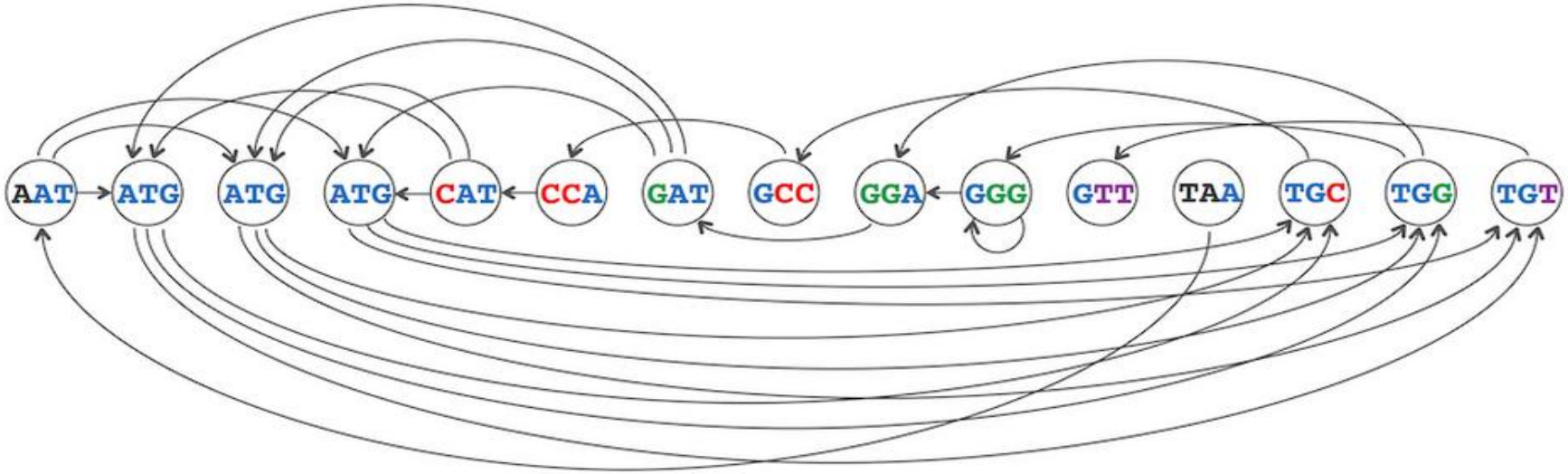


Figure: In addition to the Hamiltonian path that reconstructs **TA**ATGCCATGGGATGTT, another Hamiltonian path in the overlap graph spells the genome **TA**ATGGGATGCCATGTT. These two genomes differ by exchanging the positions of CC and GG but have the same 3-mer composition.

Construct a Hamiltonian path in a graph

- **Input:** A directed graph.
- **Output:** A path visiting every node in the graph exactly once (if such a path exists).

De-Brujin Graph for Genome Assembly

- Nicolaas de Bruijn, a Dutch mathematician (1946) was interested in solving a purely theoretical problem
- A binary string is a string composed only of 0's and 1's; a binary string is k-universal if it contains every binary k-mer exactly once
- For example, 0001110100 is a 3-universal string, as it contains each of the eight binary 3-mers (000, 001, 011, 111, 110, 101, 010, and 100) exactly once
- Finding a *k*-universal string is equivalent to solving the String Reconstruction Problem when the *k*-mer composition is the collection of all binary *k*-mers. Thus, finding a *k*-universal string can be reduced to finding a Hamiltonian path in the overlap graph formed on all binary *k*-mers

K-universal string construction

- de Bruijn was interested in constructing *k*-universal strings for arbitrary values of *k*
- For example, to find a 20-universal string, you would have to consider a graph with over a million nodes
- It is absolutely unclear how to find a Hamiltonian path in such a huge graph, or even whether such a path exists!

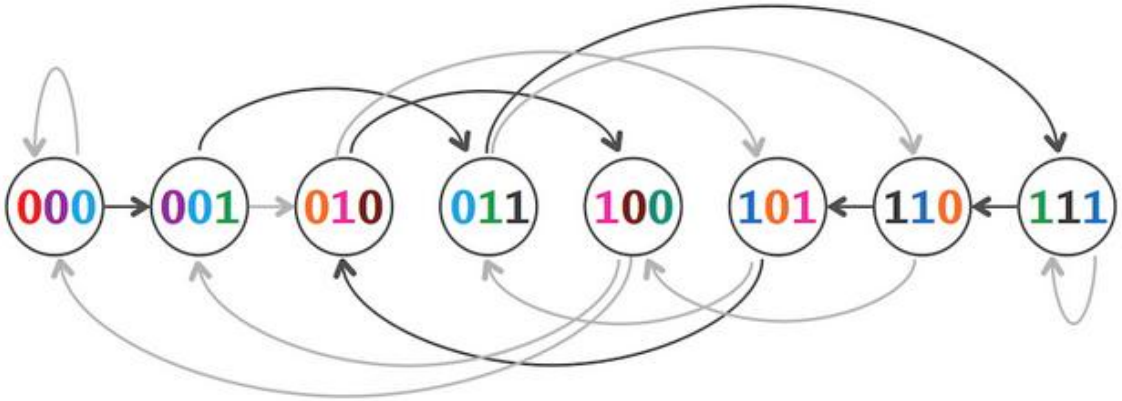


Figure: A Hamiltonian path (connecting node 000 to 100) in the overlap graph of all binary 3-mers

Gluing nodes and de Bruijn graphs

- Let's us represent the genome **TA**ATGCCATGGGATGTT as a sequence of its 3-mers:

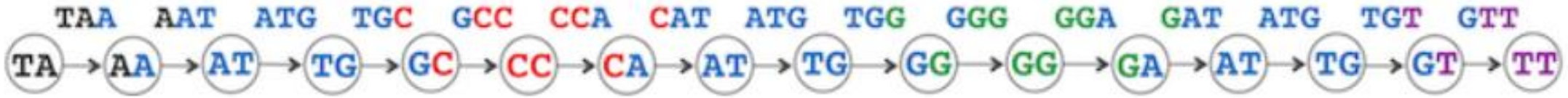
TAA AAT ATG TGC GCC CCA CAT ATG TGG GGG GGA GAT ATG TGT GTT

- instead of assigning these 3-mers to *nodes*, we will assign them to *edges*, as shown

Figure: Genome **TA**ATGCCATGGGATGTT represented as a path, with edges (rather than nodes) labeled by 3-mers.

- We can construct the genome from left to right by adding one new nucleotide to each step

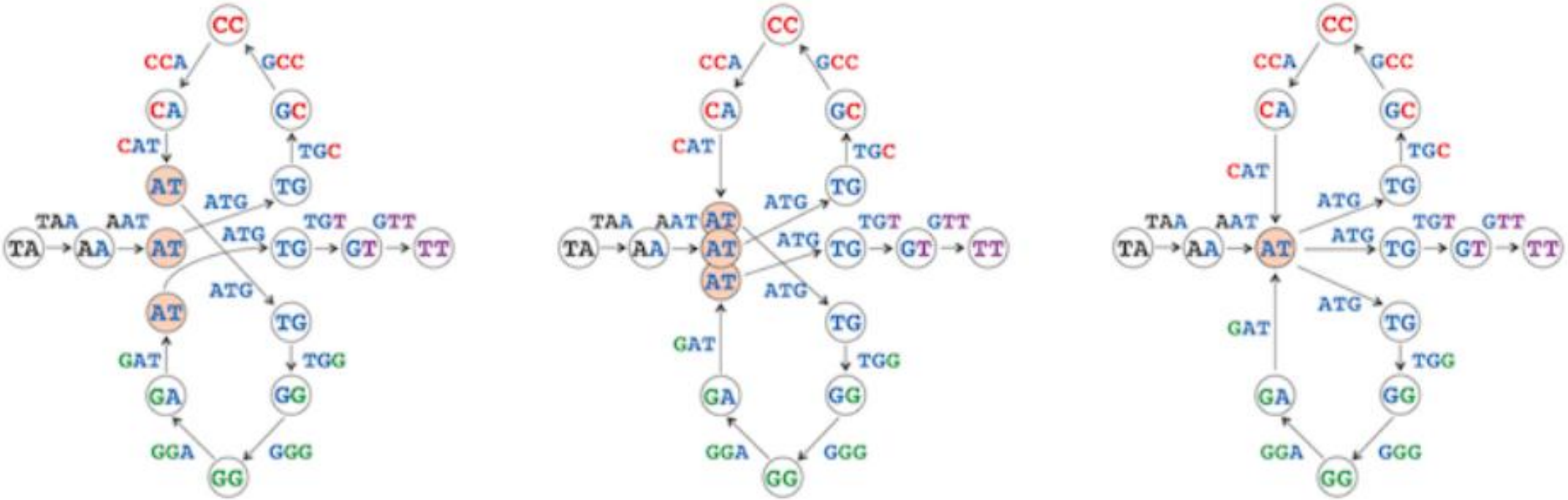
- Since each pair of consecutive edges represents consecutive 3-mers that overlap in two nucleotides, we will label each node of this graph with 2-mer representing the overlapping nucleotides shared by the edges on either side of the node e.g.



The node with incoming edge CAT and outgoing edge ATG is labeled AT

Continued..

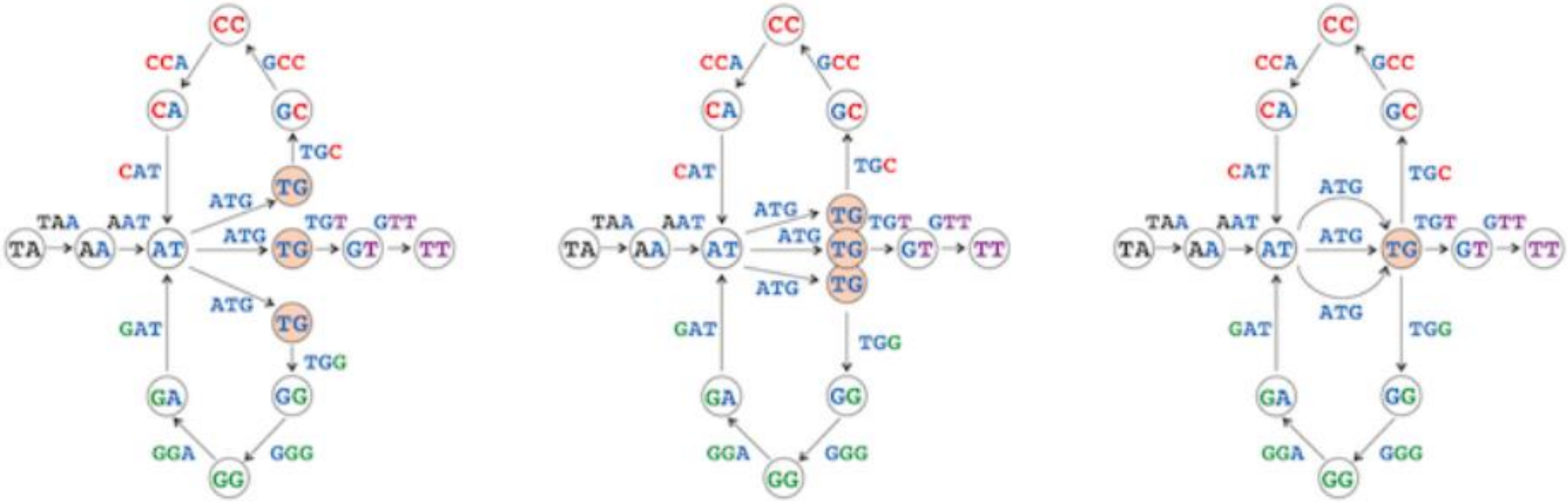
There is nothing new until we start **gluing** identically labeled nodes i.e. the figure below, we bring the three AT nodes closer and closer to each other until they have been glued into a single node.



Bringing the three nodes labeled AT closer (left) and closer (middle) to each other to eventually glue them into a single node (right). The path with sixteen nodes from the previous step has been transformed into a graph with fourteen nodes

Continued..

Note that there are also three nodes labeled by TG, which we bring closer and closer to each other in the figure below until they are glued into a single node



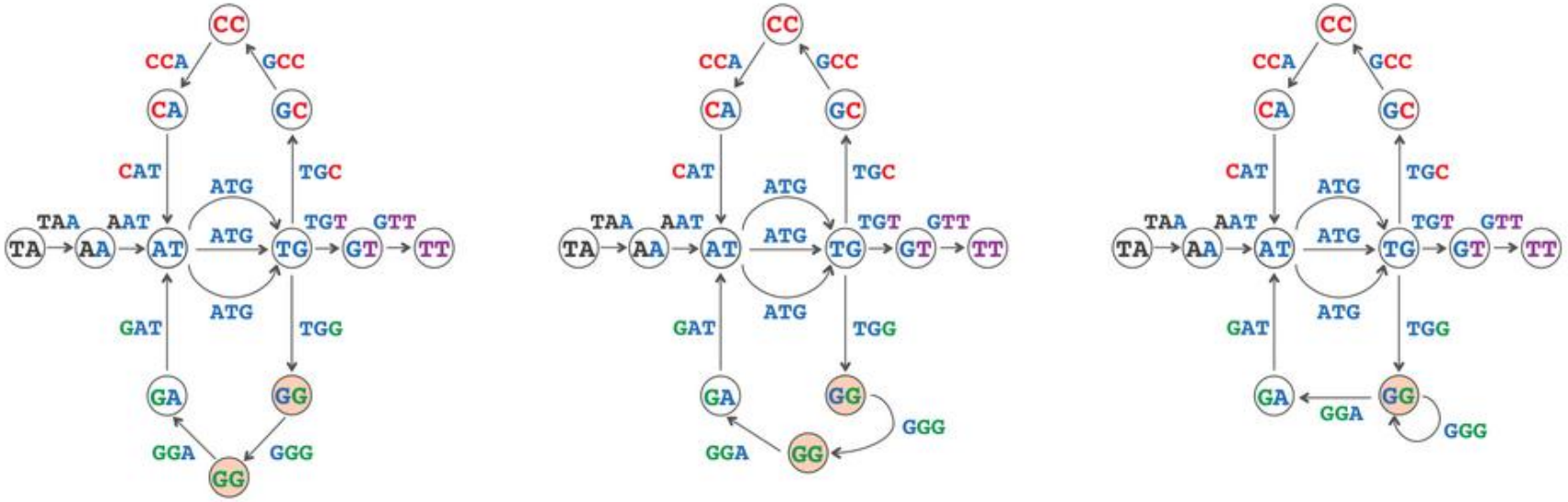
Bringing the three nodes labeled TG closer (left) and closer (middle) to each other to eventually glue them into a single node (right). The path with 16 nodes has been transformed into a graph with twelve nodes.

Continued…

Finally, we glue together the two nodes labeled GG (**GG** and **GG**), which produces a special type of edge called a **loop** connecting GG to itself

The number of nodes in the resulting graph (shown on the right below) has reduced from sixteen to eleven, while the number of edges stayed the same. This graph is called the **de Bruijn graph** of **TAATGCCATGGGATGTT**, denoted *DeBruijn*$_3$(**TAATGCCATGGGATGTT**)

Note that the de Bruijn graph below has three different edges connecting **AT** to **TG** (representing three copies of the repeat **ATG**)



Continued..

- In general, given a genome *Text*, $PathGraph_k(Text)$ is the path consisting of $|Text| - k + 1$ edges, where the i-th edge of this path is labeled by the i-th k-mer in *Text* and the i-th node of the path is labeled by the i-th $(k - 1)$-mer in *Text*. The **de Bruijn graph** $DeBruijn_k(Text)$ is formed by gluing identically labeled nodes in $PathGraph_k(Text)$.

Construct the de Bruijn graph of a string

- **Input**: An integer k and a string *Text*
- **Output**: $DeBruijn_k(Text)$

Solve the De Bruijn Graph from a String Problem

- **Input**: An integer k and a string *Text*.
- **Output**: $DeBruijn_k(Text)$, in the form of an adjacency list

How does the graph $DeBruijn_3$(**TAATGCCATGGGATGTT**) compare to $DeBruijn_3$(**TAATGGGATGCCATGTT**)?

Walking in the de Bruijn Graph

Eulerian paths

- Though we have glued together nodes to form the de Bruijn graph, we have not changed its edges, and so the path from **TA** to **TT** reconstructing the genome is still hiding in *DeBruijn*$_3$(**TAATGCCATGGGATGTT**)

- Therefore, solving the String Reconstruction Problem reduces to finding a path in the de Bruijn graph that visits every *edge* exactly once. Such a path is called a **Eulerian Path**

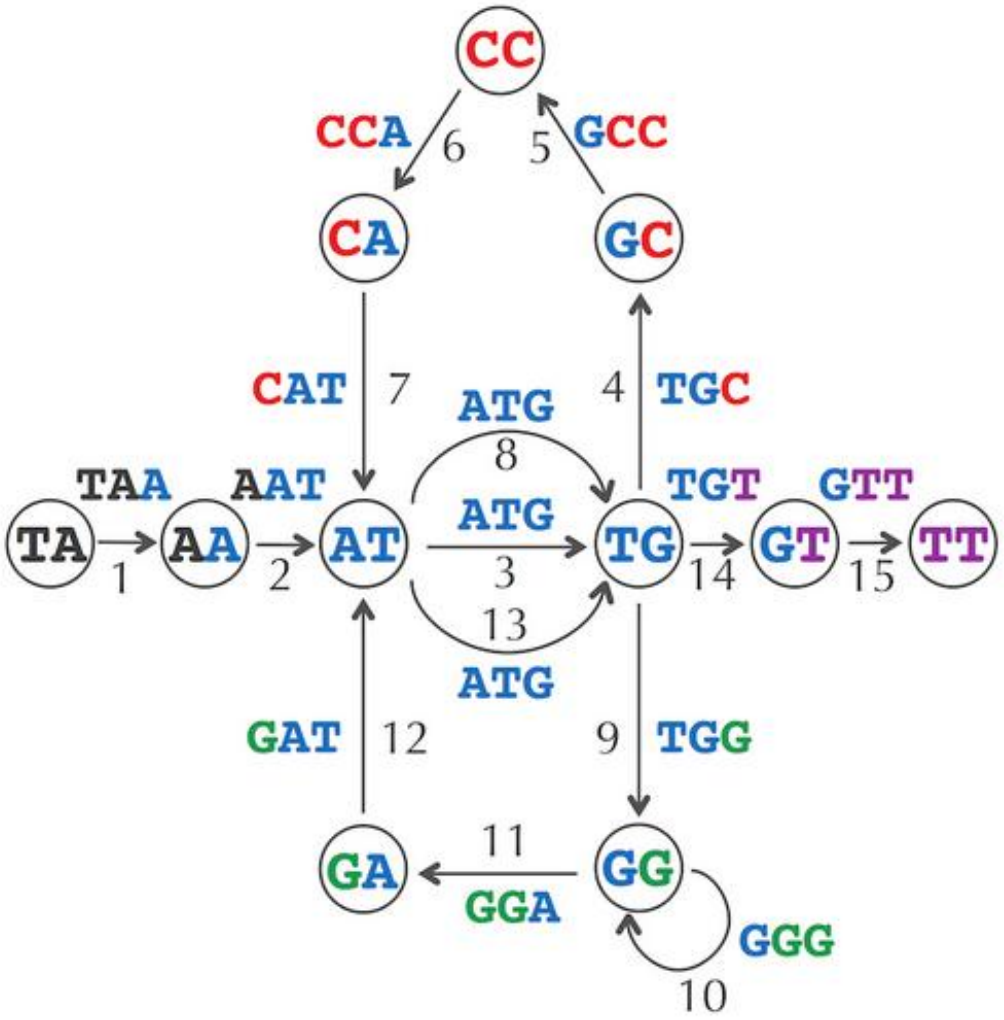


Figure: The path from **TA** to **TT** spelling out the genome **TA**ATGCCATGGGATGTT has become "tangled" in the de Bruijn graph. The numbering of the fifteen edges of the path indicates an Eulerian path reconstructing the genome.

Eulerian Path Problem

Construct an Eulerian path in a graph.

- **Input:** A directed graph.
- **Output:** A path visiting every edge in the graph exactly once (if it exists).

Another way to construct de Bruijn graphs

- The given figure represents the 3-mer composition of **TAATGCCATGGGATGTT** as a composition graph *CompositionGraph*$_3$(**TAATGCCATGGGATGTT**)
- As with the de Bruijn graph, each 3-mer is assigned to a directed edge, with its prefix labeling the first node of the edge and its suffix labeling the second node of the edge
- However, the edges of this graph are isolated, meaning that no two edges share a node

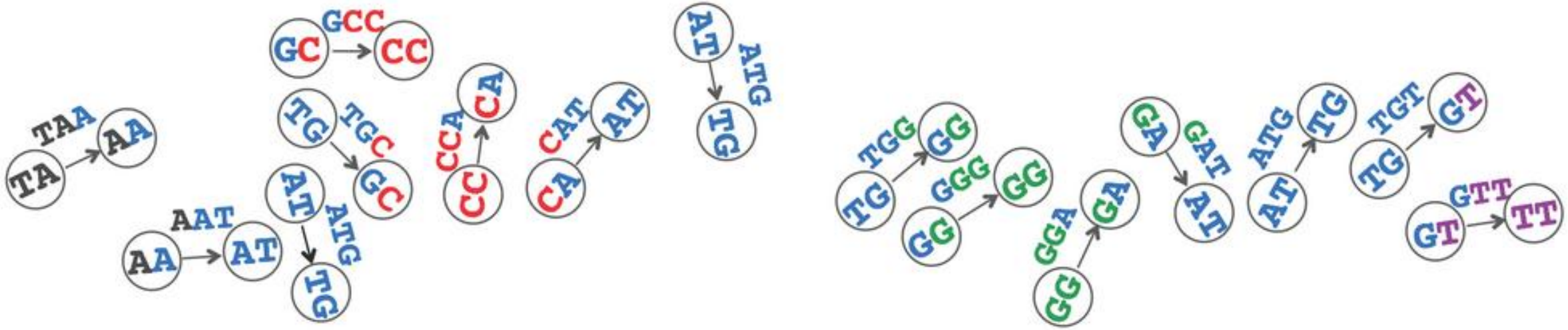


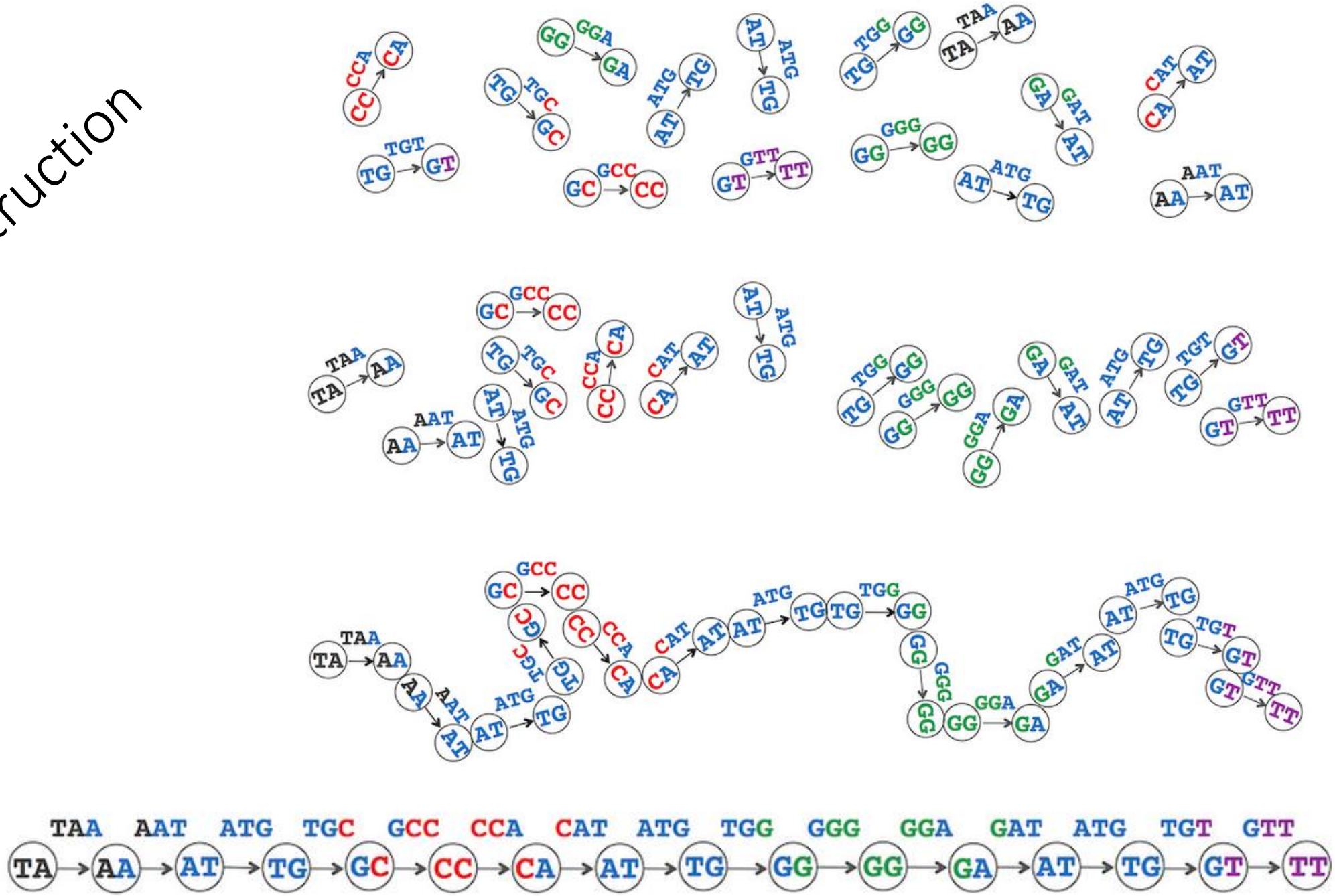
Figure: *CompositionGraph*$_3$(**TAATGCCATGGGATGTT**) represents the 3-mer composition of **TAATGCCATGGGATGTT** using an isolated directed edge for each 3-mer.

STOP and Think

- Glue identically labeled nodes in *CompositionGraph*$_3$(**TA**ATGCCATGGGATGTT)
- How does the resulting graph compare to *DeBruijn*$_3$(**TA**ATGCCATGGGATGTT)?

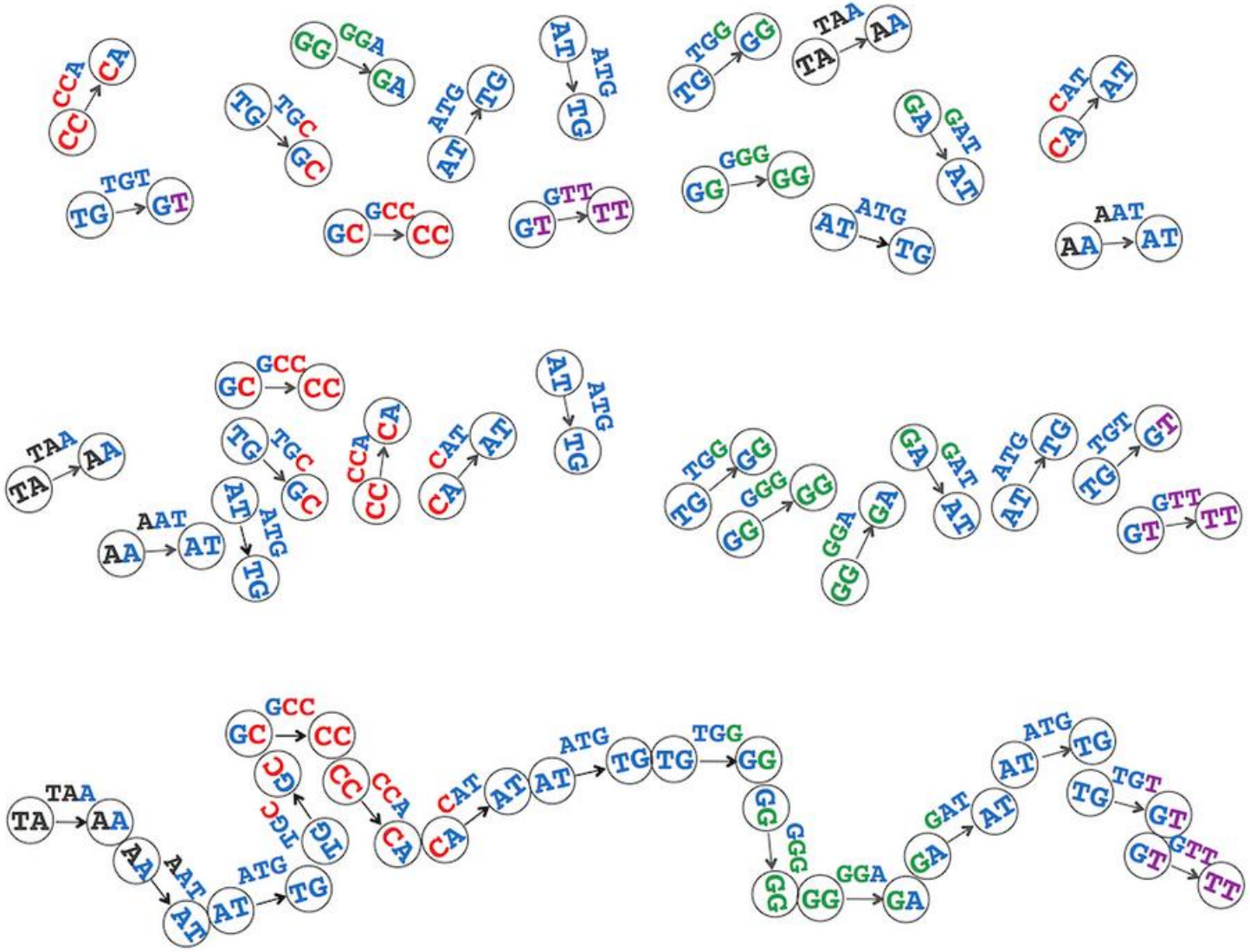
- The figure shows how *CompositionGraph*$_3$(**TA**ATGCCATGGGATGTT) changes after gluing nodes with the same label
- These operations glue the fifteen isolated edges in *CompositionGraph*$_3$(**TA**ATGCCATGGGATGTT) into a path *PathGraph*$_3$(**TA**ATGCCATGGGATGTT)
- Follow-up gluing operations proceed in exactly the same way as when we glued nodes of *PathGraph*$_3$(**TA**ATGCCATGGGATGTT), which results in *DeBruijn*$_3$(**TA**ATGCCATGGGATGTT)
- Thus, we can construct the de Bruijn graph from this genome's 3-mer composition without knowing the genome!

Reconstruction



STOP and Think

- From the previous step, we reproduced graph, we identified ATG and TGC as overlapping 3-mers
- In reality, since the genome is unknown, we don't know whether this ATG is followed by TGC, TGG, or TGT
- What would happen if we had identified ATG and TGG as overlapping 3-mers instead? Verify that the final result will be the same de Bruijn graph that we derived earlier



Continued..

- Given an arbitrary collection of *k*-mers *Patterns* (where some *k*-mers may appear multiple times), we define *CompositionGraph*(*Patterns*) as a graph with |*Patterns*| isolated edges.
- Every edge is labeled by a *k*-mer from Patterns, and the starting and ending nodes of an edge are labeled by the prefix and suffix of the *k*-mer labeling that edge.
- We then define *DeBruijn*(*Patterns*) by gluing identically labeled nodes in *CompositionGraph*(*Patterns*), which yields the following algorithm.

DeBruijn(*Patterns*)

dB ← graph in which every *k*-mer in *Patterns* is isolated edge between its prefix and suffix

dB ← graph resulting from gluing all nodes in *dB* with identical labels

return *dB*

Constructing de Bruijn graphs from *k*-mer composition

- Given a collection of *k*-mers *Patterns*, the nodes of $DeBruijn_k(Patterns)$ are simply all unique (k–1)-mers occurring as a prefix or suffix in *Patterns*. For example, say we are given the following collection of 3-mers:

  AAT ATG ATG ATG CAT CCA GAT GCC GGA GGG GTT TAA TGC TGG TGT

- Then the set of eleven *unique* 2-mers occurring as a prefix or suffix of 3-mers in this collection is as follows:

  AA AT CA CC GA GC GG GT TA TG TT

- For every *k*-mer in *Patterns*, we connect its prefix node to its suffix node by a directed edge in order to produce *DeBruijn*(*Patterns*)

Verify!!!!!!!!!!!

Continued..

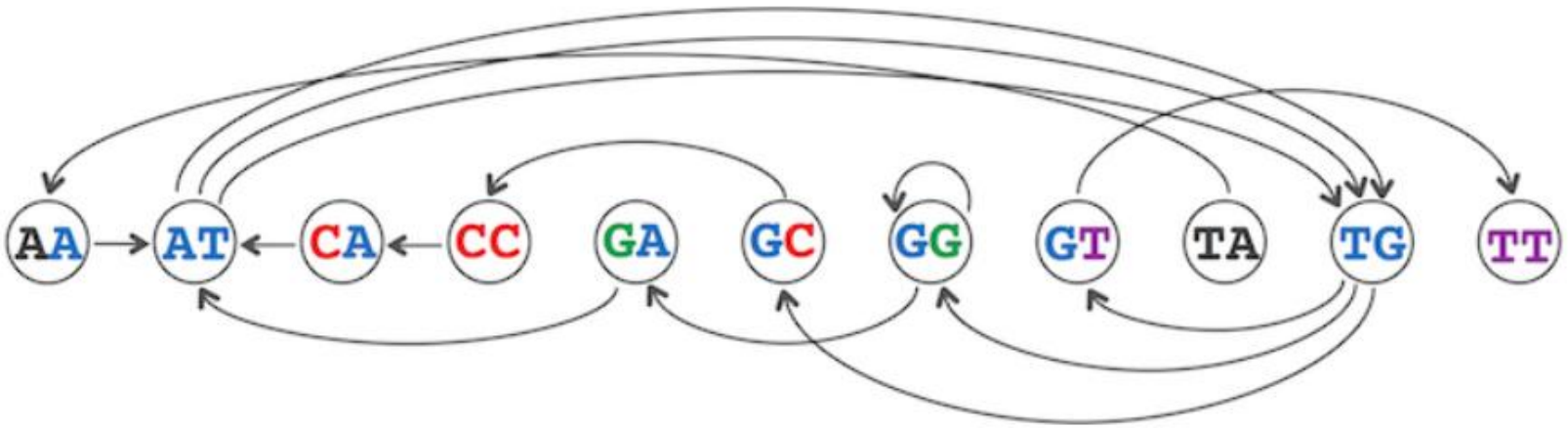


Figure: The graph above is the de Bruijn graph we have been working with, although we have drawn it differently.

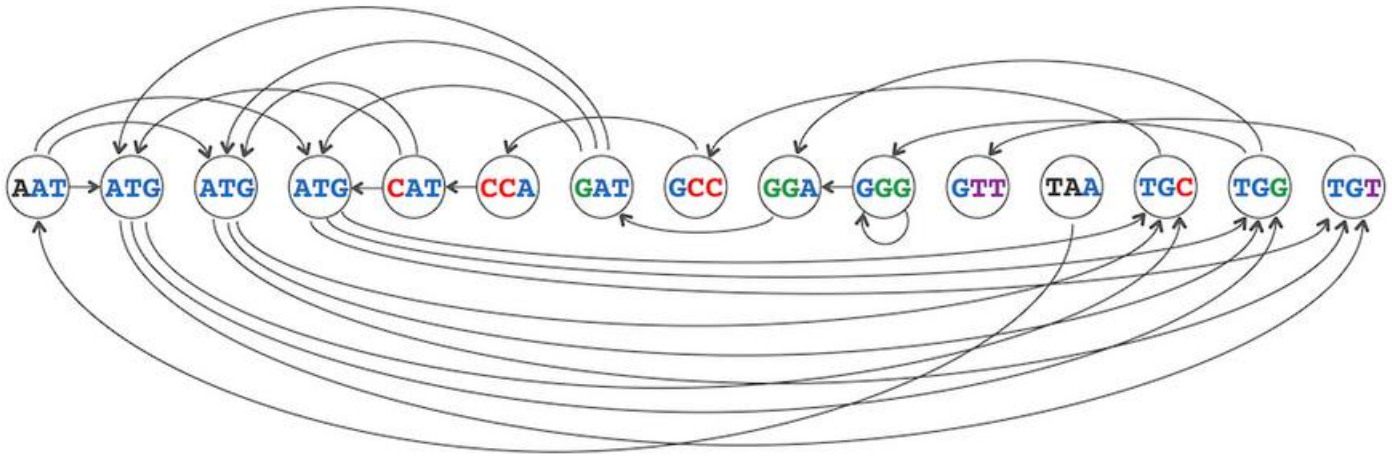
DeBruijn Graph from *k*-mers Problem: *Construct the de Bruijn graph from a set of k-mers.*

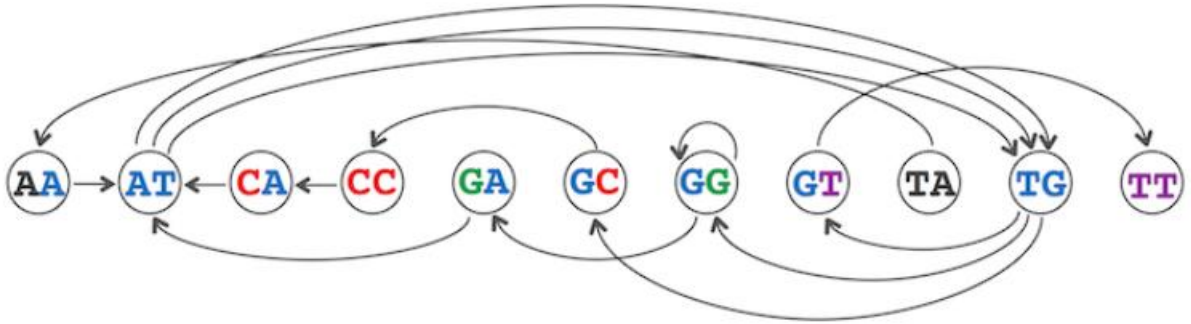
- **Input:** A collection of *k*-mers *Patterns*.
- **Output:** The adjacency list of the de Bruijn graph *DeBruijn*(*Patterns*).

De Bruijn graphs versus overlap graphs

- We have two ways of solving the String Reconstruction Problem i.e. find a Hamiltonian path in the overlap graph or find an Eulerian path in the de Bruijn graph
- Was it really worth my time to learn two slightly different ways of solving the same problem?

If we can find an efficient algorithm for the Hamiltonian Path Problem, but not for the Eulerian path Problem, then you should select the overlap graph even though it looks more complex.

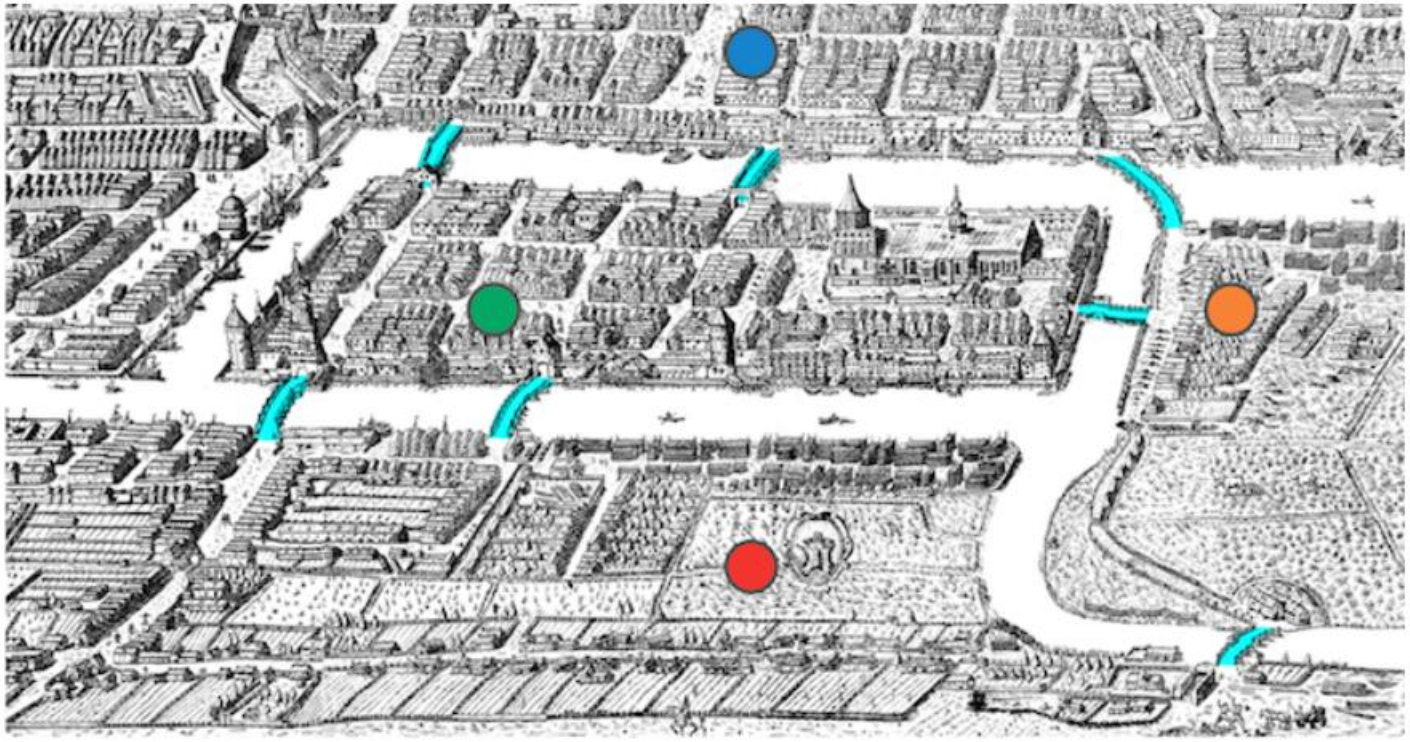




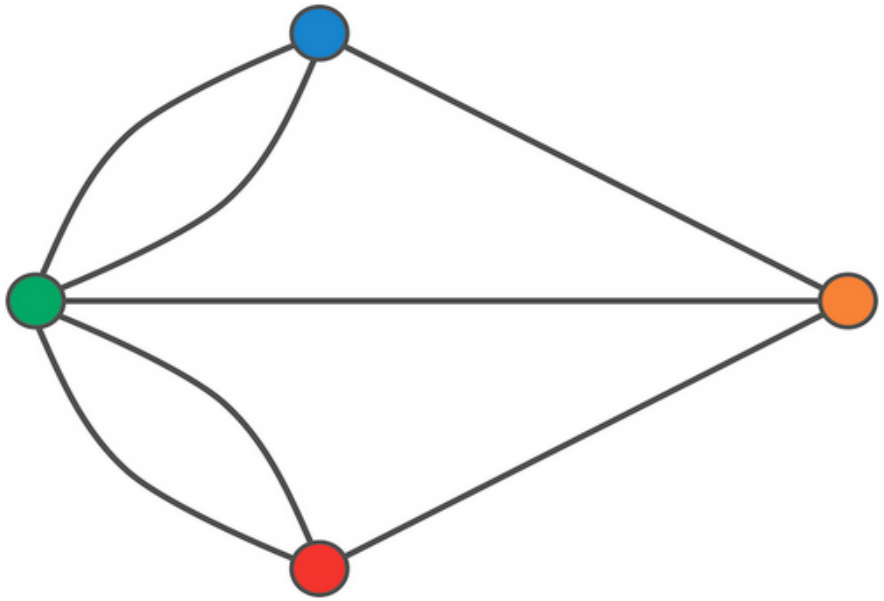
Which graph would you rather work with, the overlap graph or the de Bruijn graph?

The Seven Bridges of Konigsberg

- The Prussian city of Königsberg and this city, which today is Kaliningrad, Russia, comprised both banks of the Pregel River as well as two river islands; seven bridges connected these four different parts of the city



A map of Königsberg, adapted from Joachim Bering's 1613 illustration. The city was made up of four sectors represented by the blue, red, yellow, and green dots



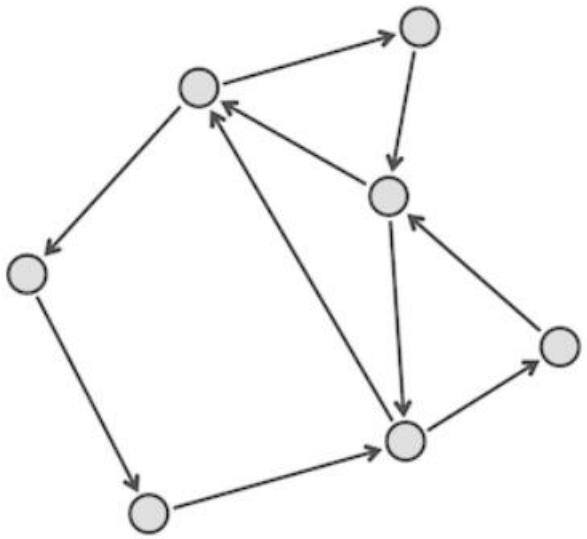
This graph's nodes represent the four sectors of the city, and its edges represent the seven bridges connecting different sectors. Note that the edges of *Königsberg* are undirected, meaning that they can be traversed in either direction

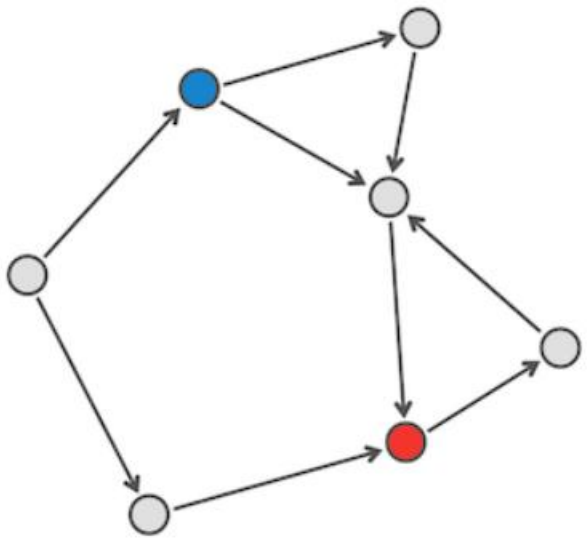
Redefine the Bridges of Königsberg Problem as a question about the graph *Königsberg*

- Eulerian path as a path in a graph traversing each edge of a graph exactly once. A cycle that traverses each edge of a graph exactly once is called an **Eulerian cycle**, and a graph containing such a cycle is **Eulerian**
- Euler solved the Bridges of Königsberg Problem, showing that no walk can cross each bridge exactly once (i.e., the graph *Königsberg* is not Eulerian)
- His theorem immediately implies an efficient algorithm for constructing an Eulerian cycle in any Eulerian graph, even one having millions of edges
- Computer scientists classify an algorithm as **polynomial** if its running time can be bounded by a polynomial in the length of the input data. On the other hand, an algorithm is exponential if its runtime on some datasets is **exponential** in the length of the input data

Euler's Theorem

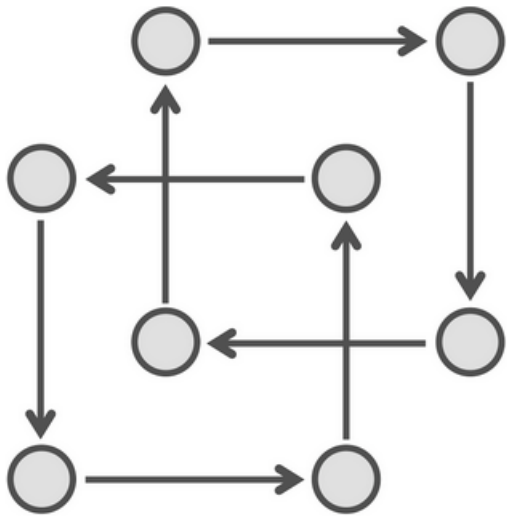
- Euler worked with undirected graphs like *Königsberg*, but we will consider an analogue of his algorithm for directed graphs so that his method will apply to genome assembly
- In order for a graph to be Eulerian, the number of incoming edges at any node must be equal to the number of outgoing edges at that node
- We define the **indegree** and **outdegree** of a node *v* (denoted *in*(*v*) and *out*(*v*), respectively) as the number of edges leading into and out of *v*
- A node *v* is **balanced** if *in*(*v*) = *out*(*v*), and a graph is **balanced** if all its nodes are balanced





STOP and Think: We now know that every Eulerian graph is balanced; is every balanced graph Eulerian?

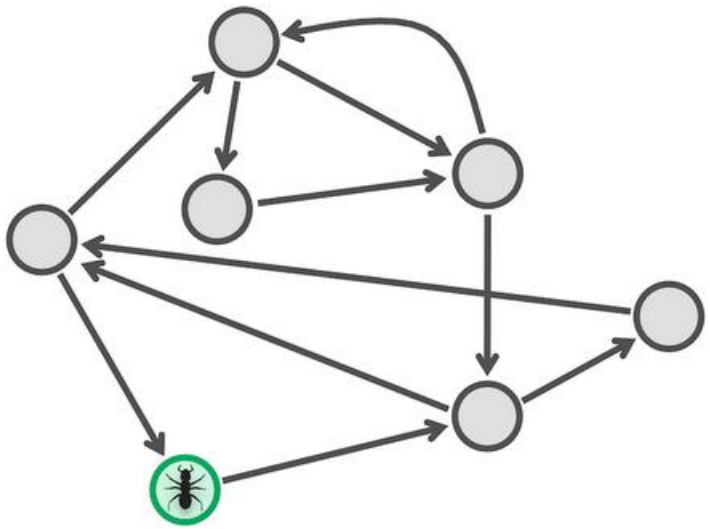
- The graph in the figure below is balanced but not Eulerian because it is **disconnected**, meaning that some nodes cannot be reached from other nodes
- In any disconnected graph, it is impossible to find an Eulerian cycle. In contrast, we say that a directed graph is **strongly connected** if it is possible to reach any node from every other node
- Euler's Theorem states that these two conditions are sufficient to guarantee that an arbitrary graph is Eulerian



To prove that *Graph* has an Eulerian cycle, place Leo at any node v_0 of *Graph* (the green node in the figure above), and let him randomly walk through the graph under the condition that he cannot traverse the same edge twice

If Leo were incredibly lucky — or a genius — then he would traverse each edge exactly once and return back to v_0. However, odds are that Leo will “get stuck" somewhere before he can complete an Eulerian cycle, meaning that he reaches a node and finds no unused edges leaving that node

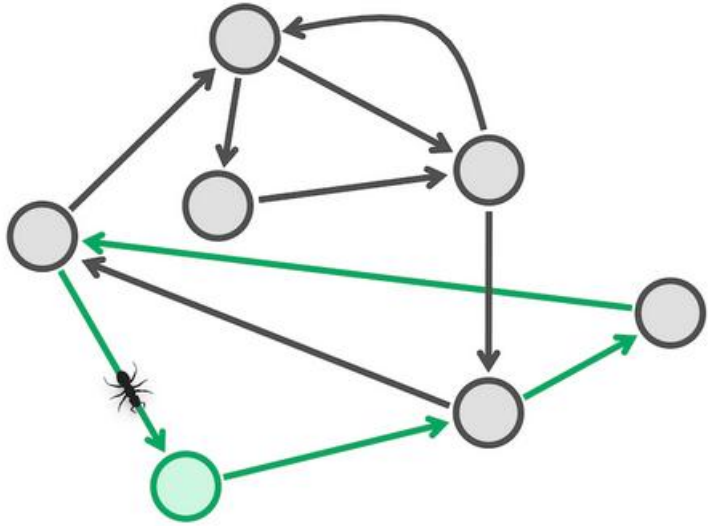
Lets starts at the green node v_0 and walks through a balanced and strongly connected graph.



Continued..

- Where is Leo when he gets stuck? Can he get stuck in any node of the graph or only in some nodes?
- It turns out that the only node where Leo can get stuck is the starting node v_0! The reason why is that *Graph* is balanced — if Leo walks into any node other than v_0 (through an incoming edge), then he will always be able to escape via an unused outgoing edge.

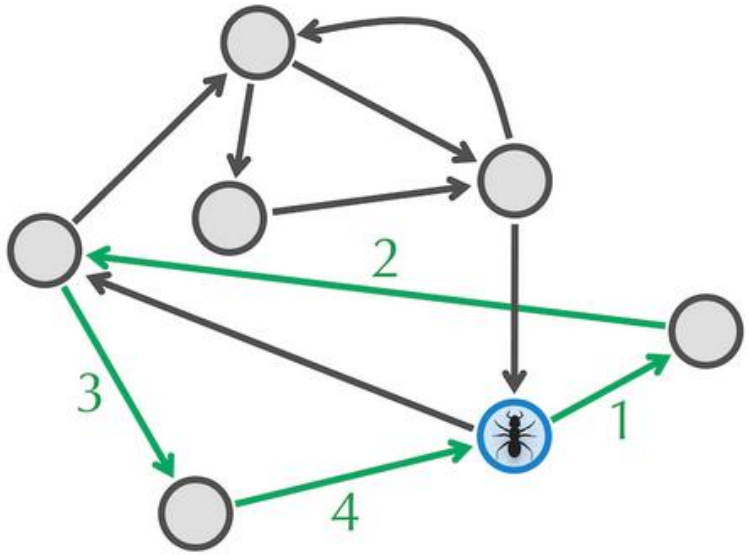
The only exception to this rule is the starting node v_0, since Leo used up one of the outgoing edges of v_0 on his first move. Now, because Leo has returned to v_0, the result of his walk was a cycle, which we call $Cycle_0$



Leo produces a cycle Cycle_0 (formed by green edges) when he gets stuck at the green node v_0. In this case, he has not yet visited every edge in the graph.

Continued..

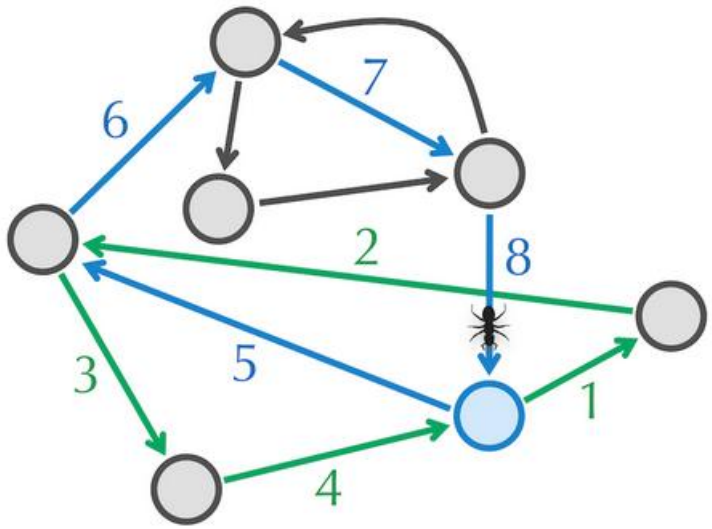
- Is there a way to give Leo different instructions so that he selects a longer walk through the graph before he gets stuck?
- As we mentioned, if $Cycle_0$ is Eulerian, we are finished. Otherwise, because *Graph* is strongly connected, some node on $Cycle_0$ must have unused edges entering it and leaving it. Naming this node v_1 , we ask Leo to start at v_1 instead of v_0 and traverse $Cycle_0$ (thus returning to v_1)



Starting at a new node v_1 (shown in blue), Leo first travels along $Cycle_0$, returning to v_1. Note that v_1, unlike v_0, has unused outgoing and incoming edges.

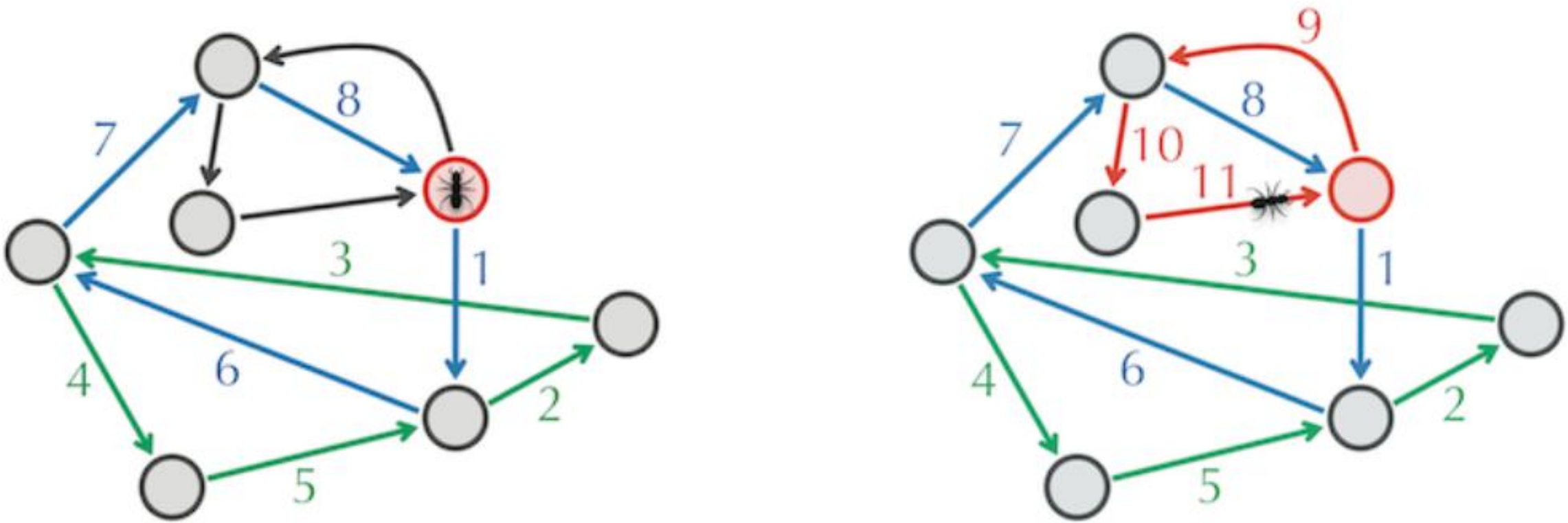
Continued..

- Leo is probably annoyed that we have asked him to travel along the exact same cycle, since as before, he will eventually get stuck at v_1, the node where he started. However, now there are unused edges starting at this node, and so he can continue walking from v_1, using a new edge each time. The same argument as the one that we used above implies that Leo must eventually get stuck at v_1. The result of Leo's walk is a new cycle, $Cycle_{1,}$ that is larger than $Cycle_0$.



After traversing the previously constructed green cycle $Cycle_0$, Leo continues walking and eventually produces a larger cycle $Cycle_1$ formed of both the green and the blue cycles put together into a single cycle.

Continued..



(Left) Starting at a new node v_2 (shown in red), the ant first travels along the previously constructed $Cycle_1$ (shown as green and blue edges).

(Right) After completing the walk through $Cycle_1$, the Leo continues randomly walking through the graph and finally produces an Eulerian cycle.

Continued..

- If $Cycle_1$ is an Eulerian cycle, then Leo has completed his job. Otherwise, we select a node v_2 in $Cycle_1$ that has unused edges entering it and leaving it (the red node in the figure above).
- Starting at v_2, we ask Leo to traverse $Cycle_1$, returning to v_2 (right panel above). Afterwards, Leo will randomly walk until gets stuck at v_2, creating an even larger cycle namely $Cycle_2$.
- $Cycle_2$ is Eulerian, although this is certainly not the case for an arbitrary graph. In general, Leo generates larger and larger cycles at each iteration, and so we are guaranteed that $Cycle_m$ will traverse all the edges in *Graph*. This cycle must be Eulerian, and so we (and Leo) are finished.
- Can you formulate and prove an analogue of Euler's Theorem for undirected graphs?